

UECS1044 UECS1144 Object-Oriented Application Development

Group Assignment (June 2026)

Smart Metro Ticketing System

Release of group assignment:

Friday 03 July 2026

Project Deadline:

Week 13: Friday 11 September 2026

1 Instruction

- This is a **group project**. Each group should consist of 5 to 6 students.
- Each group must develop a console-based Java application using Object-Oriented Programming principles.
- The final project must be submitted as a ZIP file. The ZIP file name should follow this format: G1_CS_UECSXXXX_GA.zip
- Each group must meet the lecturer at least three times during the project development process.
- Please ensure that you are logged into your UTAR student portal on your browser before attempting to access this link to specify your assignment group:
 - <https://tinyurl.com/bddh7snm>

2 Requirements

The project must be implemented using:

- Java
- Console-based interface
- Object-Oriented Programming
- TXT file handling
- **Optional:** JSON or XML file handling Project details

The project must include:

- Class and Object
- Inheritance
- Polymorphism
- Abstract Class or Interface
- Exception Handling
- Collections: ArrayList and HashMap
- File Handling
- Enum
- Package.

3 Project Details

A. Introduction

This assignment is a team project aimed at developing a **Smart Metro Ticketing System** using Java.

The system should simulate the basic operations of a metro ticketing application. It should allow passengers to register, log in, buy tickets, cancel tickets, make payments, and view ticket information. It should also allow the admin to manage stations, trains, routes, tickets, and reports.

The project should be developed as a menu-driven console application. Students are expected to design proper classes, apply inheritance and polymorphism, handle exceptions, store data using collections, and save/load data using files.

B. Problem Specification

The objective of this project is to develop a console-based Java application called Smart Metro Ticketing System. The system should simulate the basic operations of a metro ticketing system,

including user registration, login, station management, train management, route management, ticket booking, fare calculation, payment processing, report generation, and file-based data storage.

The system must support two main types of users: Passenger and Admin. Both Passenger and Admin should inherit from an abstract User class. A passenger can register, log in, view profile information, top up balance, buy tickets, cancel tickets, and view ticket information. An admin can manage stations, trains, routes, users, and reports.

The system must allow the admin to add and view metro stations. Each station should have a station ID, name, and location. The system should also allow searching for a station by name. The admin should be able to add train information, including train ID, train name, and capacity.

The system must allow the admin to create routes between two stations. Each route should contain a route ID, source station, destination station, and distance in kilometers. The route information will be used later for fare calculation and ticket booking.

The ticket booking module should allow a passenger to buy a ticket by selecting a source station, destination station, and ticket type. The system should calculate the fare automatically based on the selected route and ticket type. Each ticket should contain ticket ID, passenger information, source station, destination station, ticket type, ticket status, and fare amount.

The system should support different ticket types such as single ticket, daily ticket, and monthly ticket. It should also support ticket statuses such as active, used, and cancelled. These fixed values must be implemented using enum types.

The fare calculation should be designed using an interface called FareCalculator. A class called StandardFareCalculator should implement this interface and calculate the fare based on route distance and ticket type. This design allows students to practice interface-based programming and polymorphism.

The payment module should simulate at least two payment methods: cash payment and card payment. Both CashPayment and CardPayment should implement a common Payment interface. The system should process payment through the PaymentService class. This part of the project is intended to demonstrate polymorphism.

The system must include a report module. The report module should be able to display total tickets sold, total revenue, and cancelled tickets. This helps students understand how collections can be used to process and summarize application data.

The system must use Java collections such as ArrayList and HashMap. For example, users may be stored in a HashMap using email as the key, while stations, trains, routes, and tickets may be stored in ArrayList objects.

The system must store and retrieve data using files. At minimum, students should implement TXT file handling using a FileManager interface and TXTFileManager class. JSON or XML file handling may be added as a bonus feature for stronger groups.

The system must include exception handling. It should handle invalid login attempts, ticket not found cases, and file processing errors. The application should not terminate unexpectedly when invalid input is entered. Instead, it should display clear and meaningful error messages.

The final application must be menu-driven and run in the console. The program should display clear menus for passengers and admins. The application should continue running until the user selects the exit option. Before exiting, important data should be saved into files, and when the program starts again, data should be loaded from files.

The project must demonstrate object-oriented programming principles including class and object design, inheritance, polymorphism, abstraction using interface or abstract class, encapsulation, exception handling, collections, file handling, enum, packages, Comparable or Comparator, and basic OOP principles.

Note: Any additional assumptions made by the groups to improve the program are acceptable.

C. Package Structure

scr/

app/

1. Main.java

model/

2. User.java

3. Passenger.java

4. Admin.java

5. Station.java

6. Train.java

7. Route.java

8. Ticket.java

service/

9. UserService.java

10. StationService.java

11. TrainService.java

12. RouteService.java

13. TicketService.java

14. PaymentService.java

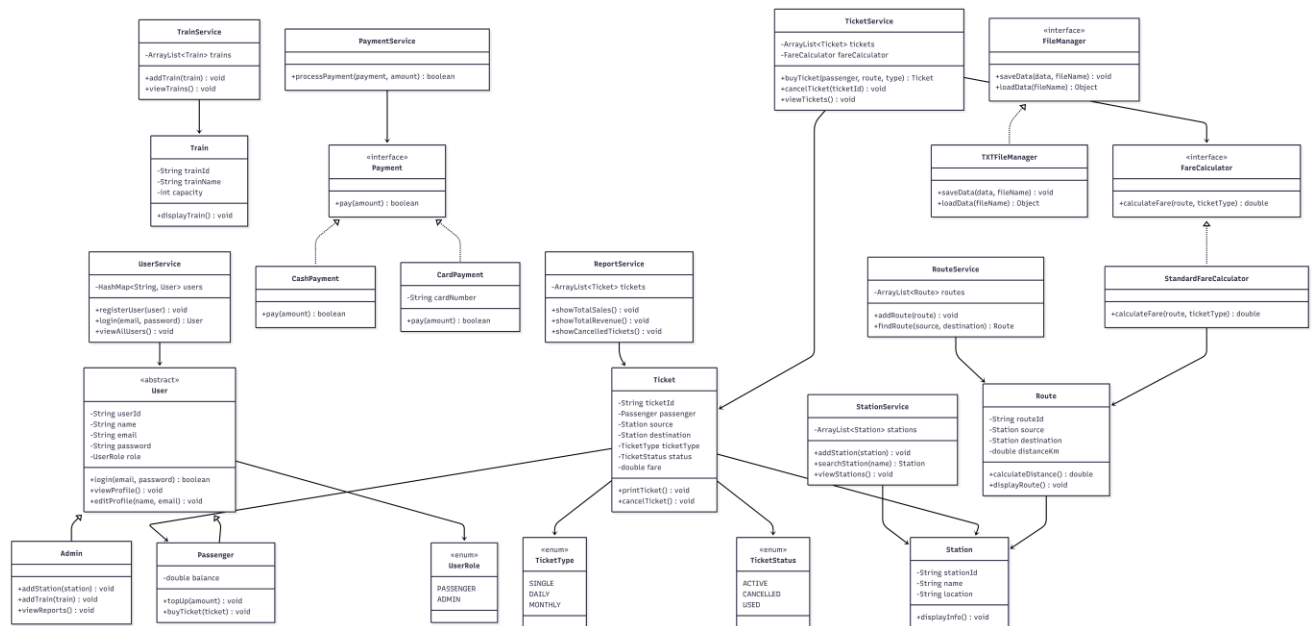
15. ReportService.java

payment/

16. Payment.java

17. CashPayment.java

28. FileProcessingException.java



4 Task

The goal of this assignment is to construct a Java-based Object-Oriented Metro Ticketing System.

The system should support the following main functions:

- Passenger registration
- User login
- View user profile
- Add station
- View stations
- Search station
- Add train
- View trains
- Create route
- View routes
- Buy ticket
- Cancel ticket
- Calculate fare
- Process payment
- Generate report
- Save data to file
- Load data from file
- Sort stations or tickets

5 Methodology

5.1 System Analysis:

Students should first analyze the system requirements and identify the main objects involved in the system.

Main objects include:

- User
- Passenger
- Admin
- Station
- Train
- Route
- Ticket
- Payment
- FareCalculator
- FileManager

5.2 Class Design

Students must design the classes based on the provided class diagram.

The project should include:

```
Abstract class:
    - User

Child classes:
    - Passenger
    - Admin

Interfaces:
    - Payment
    - FareCalculator
    - FileManager

Implementation classes:
    - CashPayment
    - CardPayment
    - StandardFareCalculator
    - TXTFileManager
```

5.3 Object-Oriented Implementation

Students must apply OOP concepts properly.

Example:

```
Inheritance:
    - User → Passenger, Admin

Polymorphism:
    - Payment payment = new CashPayment();
    - Payment payment = new CardPayment();

Interface:
    - FareCalculator
    - Payment
    - FileManager
```

5.4 Data Storage

Students must store program data using Java collections.

Example:

- `HashMap<String, User> users`
- `ArrayList<Station> stations`
- `ArrayList<Train> trains`
- `ArrayList<Route> routes`
- `ArrayList<Ticket> tickets`

The system must also support file saving and loading using TXT files.

Optional bonus:

- JSON file handling
- XML file handling

5.5 Error Handling

Students must use exception handling to prevent the system from crashing.

Required exceptions:

- `InvalidLoginException`
- `TicketNotFoundException`
- `FileProcessingException`

Examples of errors:

- Wrong email or password
- Ticket ID not found
- File cannot be opened
- Invalid input from user

5.6 Testing

Students should test the system using different scenarios.

Example test cases:

- Register a new passenger
- Login with correct password
- Login with wrong password
- Add new station
- Create route
- Buy ticket
- Cancel ticket
- Pay by cash

- Pay by card
- Save data
- Load data
- Generate report

6 Deliverables

The deliverables for this assignment are:

A. Java Source Code

The source code should include all Java files and packages.

The code must be:

- Well-organized
- Readable
- Commented
- Executable
- Menu-driven

B. Report

The report should explain the system design, implementation, testing, and results.

C. Class Diagram PDF

Students must include the provided class diagram PDF in their submission.

Download link:

<https://tinyurl.com/2mv45tp6>



D. Sample Files

Students must provide sample files used for saving and loading data.

Example:

- users.txt
- stations.txt

- trains.txt
- routes.txt
- tickets.txt

E. Individual Contribution

Each group member must clearly explain their contribution.

Example:

Student Name	Student ID	Contribution
Student 1	240001	User Management
Student 2	240002	Ticket Module
Student 3	240003	File Handling
Student 4	240004	Report and Testing

7 Report Format

The report should include the following sections:

1. Title
2. Abstract
3. Introduction
4. Problem Specification
5. System Requirements
6. Class Diagram
7. Methodology
8. Implementation
9. Testing and Results
10. Discussion
11. Conclusion
12. References
13. Appendices
14. Individual Contribution

8 Bonus Marks

Bonus marks may be awarded for:

- JSON file handling
- XML file handling
- Discount fare system
- Travel history

- Advanced report generation
- Better menu design
- Input validation

9 Graded Components Weightage

- Object-Oriented Design and Analysis (40%)
- Code Quality and Functionality (30%)
- Testing, Documentation and User Experience (15%)
- Presentation / Demonstration Video (10%)
- Competitive Mark Component (5%)

10 Marking Rubrics

1. Object-Oriented Design and Analysis (40%)

Marks	Description
0 – 10%	Very poor object-oriented design. Classes are incomplete or poorly designed. Little or no evidence of inheritance, encapsulation, interfaces, or polymorphism. UML is missing or significantly different from the implementation.
11 – 20%	Basic object-oriented implementation. Core classes are implemented but relationships are limited. Limited use of inheritance, abstraction, interfaces, collections, and exception handling. UML only partially matches the implementation.
21 – 30%	Good object-oriented design. Most required classes and relationships are correctly implemented. Appropriate use of inheritance, interfaces/abstract classes, collections, enums, and packages. UML closely follows the implementation.
31 – 40%	Excellent object-oriented design. Well-structured class hierarchy with proper abstraction, inheritance, polymorphism, encapsulation, exception handling, collections, file handling, enums, Comparable/Comparator, and good application of OOP principles. UML accurately represents the implemented system.

2. Code Quality and Functionality (30%)

Marks	Description
0 – 7%	Program does not compile, crashes frequently, or contains major functional errors. Code is poorly organized and difficult to understand.
8 – 15%	Core functionality is partially implemented. Several required features are missing or contain errors. Code organization and documentation are limited.
16 – 22%	Most required functionalities are correctly implemented. Code is modular, readable, and reasonably organized. Exception handling and file operations are mostly correct.
23 – 30%	Fully functional application with all required features implemented. Code is clean, modular, reusable, well-commented, and follows good programming practices. Program is stable and easy to maintain.

3. Testing, Documentation and User Experience (15%)

Marks	Description
0 – 3%	Very limited testing. Documentation is incomplete or missing. Console interface is confusing and lacks input validation.
4 – 7%	Basic testing performed. Documentation is acceptable but incomplete. Console menus are functional with limited validation and usability.
8 – 11%	Good testing using multiple scenarios. Documentation is clear. Console interface is user-friendly with proper input validation and meaningful messages.
12 – 15%	Comprehensive testing covering normal and exceptional cases. Documentation is complete and well-organized. Console interface is intuitive, consistent, user-friendly, and demonstrates excellent usability.

4. Presentation / Demonstration Video (10%)

Marks	Description
0 – 2%	Presentation/video is missing, poorly organized, or fails to demonstrate the system. Team members are unable to explain the implementation.
3 – 5%	Basic demonstration of the system. Only some features are presented. Explanation is limited and lacks confidence or technical depth.
6 – 8%	Good presentation demonstrating most system features. Team members explain the design, implementation, and functionality clearly with appropriate examples.
9 – 10%	Excellent presentation. The demonstration is well-structured, professional, and clearly explains the system architecture, OOP concepts, implementation, testing, and key features. All team members actively participate and answer questions confidently.

5. Competitive Mark Component (5%)

Marks	Description
0 – 1%	Meets only the minimum project requirements with little originality or improvement.
2 – 3%	Includes some useful enhancements beyond the minimum requirements, such as improved usability, additional validation, or minor extra features.
4 – 5%	Outstanding project demonstrating creativity, innovation, excellent software design, additional useful features, and a polished implementation that clearly exceeds the minimum project requirements.